

Git指南v0.1

一文包含仓库管理、分支管理、git简单命令；

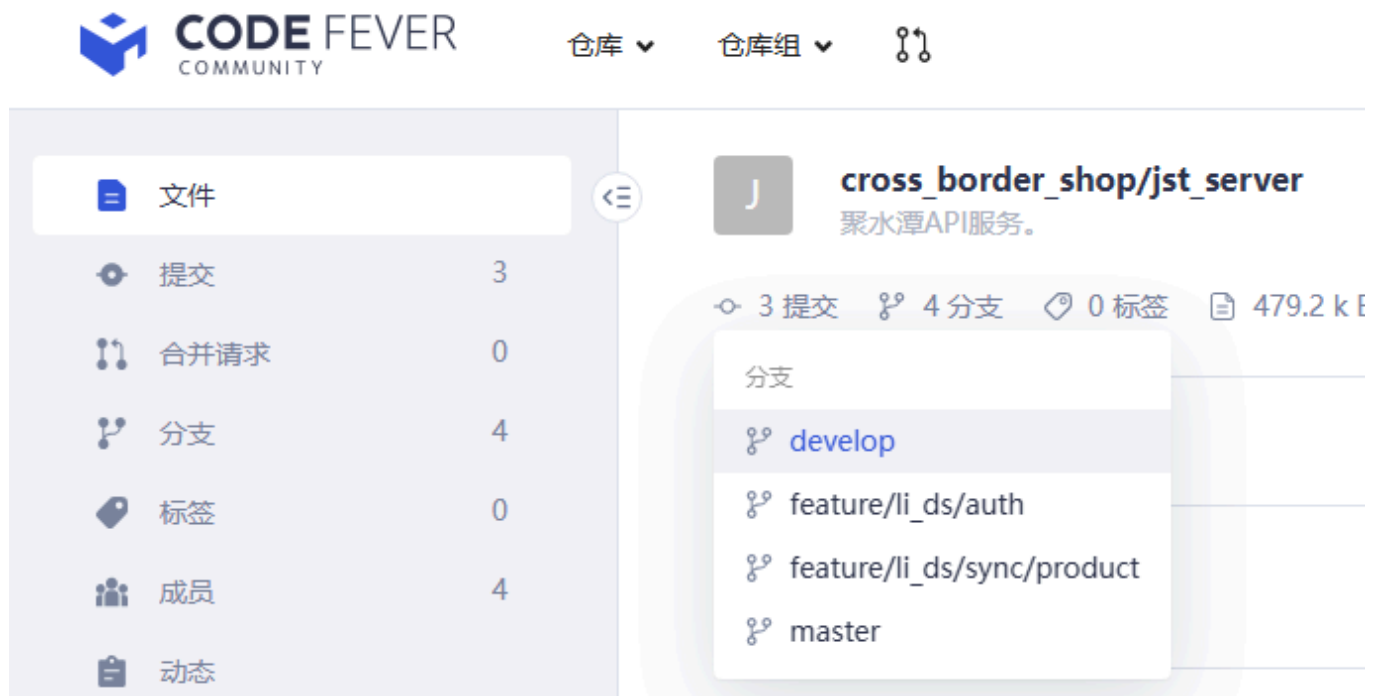
前置

IDE: Cursor

仓库管理：

待定

分支管理：



建议创建三类用途的分支；

master-稳定的线上分支，个人分支以此为基准拉取，协同开发时，应避免个人分支代码直接推送到本分支；

develop-功能最新的开发分支，个人分支应当优先推送至此分支，协同开发时，应修复所有合并冲突后，再推送至master；

个人分支-本地分支，建议命名

feature/{个人姓名缩写}/{分支用途，即开发功能}，如：

feature/li_ds/sync/product（李道爽商品同步开发分支）

或

fix/{个人姓名缩写}/{分支用途，即修复bug}，如：
fix/li_ds/product/memory/overflow（李道爽商品内存溢出修复分支）

分支全流程步骤简述：

- 1.提交并推送本地代码；
- 2.若有部分代码无需推送，使用暂存命令；
- 3.合并本地代码至develop分支；
- 4.修复冲突后合并至master分支；
- 5.确保本地仓库为最新；
- 6.切换至master分支；
- 7.拉取远程master分支最新代码；
- 8.基于当前master 创建并切换到新分支；
- 9.推送当前分支至远程仓库；
- 10.验证——远程仓库将出现你新建的个人分支；



具体命令点击自查：

[^6cea53](#)

关于提交命令的注释：

git commit -m "feat:本次提交的功能简述;"
例：

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

PS D:\Program Files\code\src\jst_server> git commit -m "feat:依据代码规范重构service层;"
```

或

`git commit -m "fix:本次提交修复的bug简述;"`

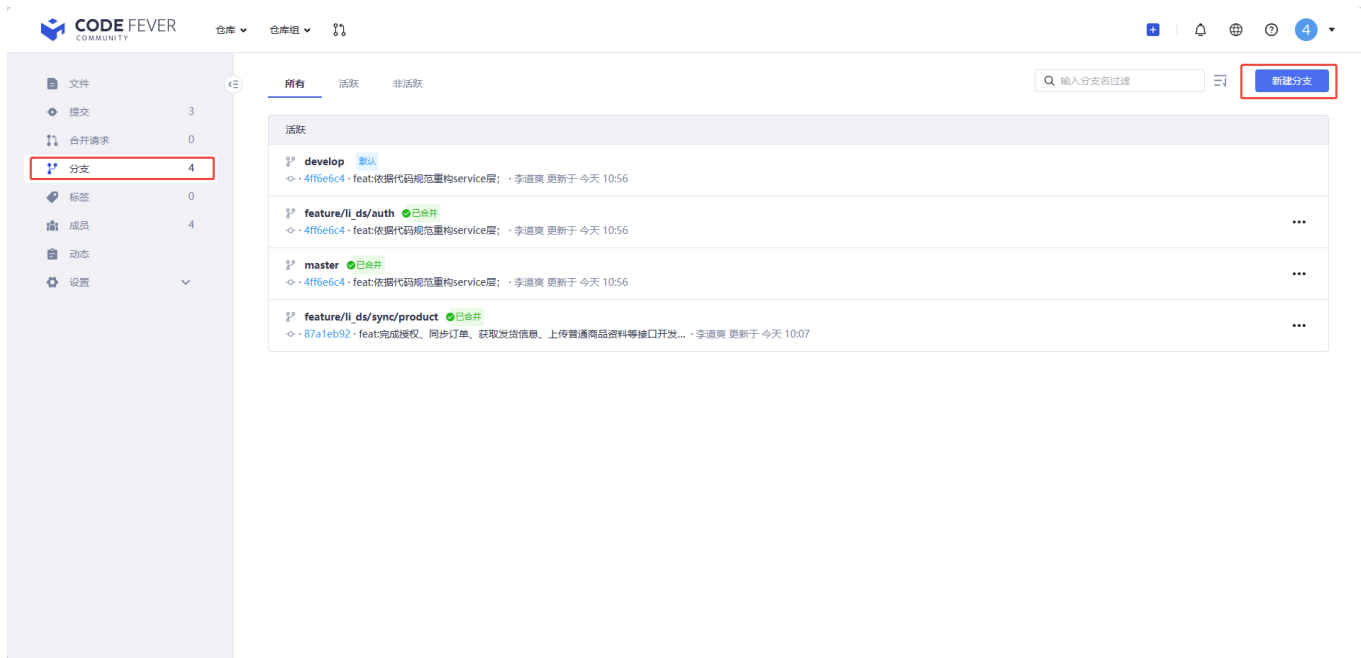
例：

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

PS D:\Program Files\code\src\jst_server> git commit -m "fix:修复erp.url缺失的bug;"
```

对应类型的注释无需限制于特定分支，即feature分支上依旧可以提交fix注释；

创建分支（网页）：



新建分支

分支名称

feature/li_ds/web/create/example

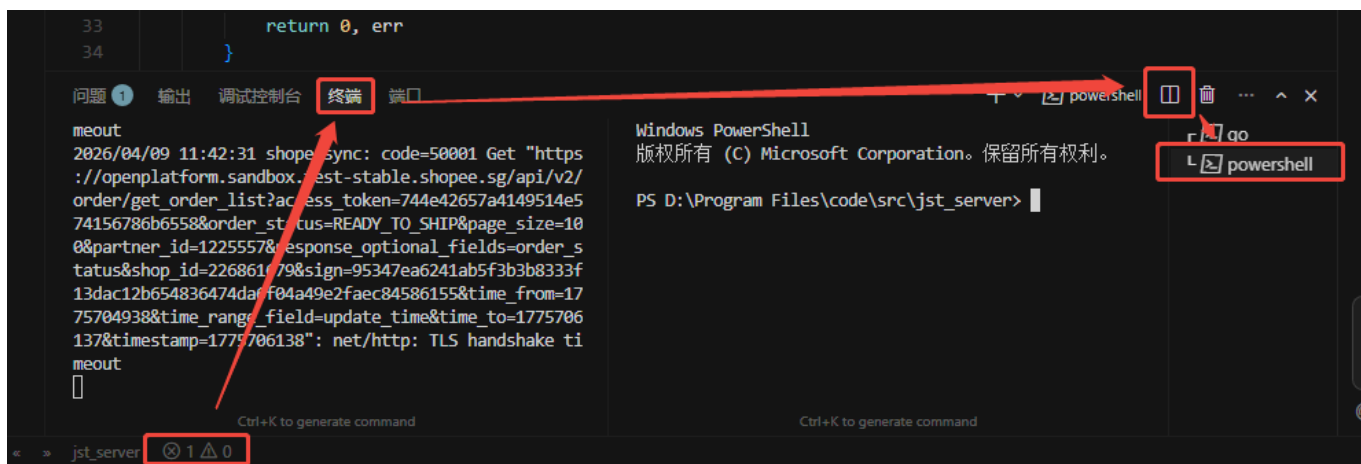
选择创建来源

master

取消

确定

创建分支 (cli) :



建议新建终端，即用即删，无需影响正在运行的本地服务；

具体命令点击自查：

[^6cea53](#)

建议参考文章：

[【Git】Git工具的正确使用指南（系统性总结）-阿里云开发者社区](#)

Git简单命令：

git项目初始化

git init

创建 .gitignore 文件，忽略编译输出和敏感文件：

内容自查

保存当前未提交的更改（包括工作区和暂存区）

git stash

确保本地仓库是最新的

git fetch origin

切换到master 分支

git checkout master

拉取远程 master 的最新代码

git pull origin master

基于当前 master 创建并切换到新分支（将 替换为你的分支名）

git checkout -b

切换分支

git checkout feature/li_ds

添加所有修改文件

git add .

提交文件

git commit -m "command"

添加远程仓库地址

git remote add origin <https://github.com/yourusername/message-forwarder.git>

创建分支

git branch -M master

推送分支

```
git push -u origin master
```